Worksheet

Computer Network - Week 9 - Thread and Deadlock

Name	Farrel Tsaqif Anindyo
Class	CSA

Activity 1

Procedure

Read the procedure on the activity file.

```
┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ touch simple_thread.c && nano simple_thread.c

┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ gcc -pthread simple_thread.c -o simple_thread.out

┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ ./simple_thread.out
Inside Thread
0
1
2
3
4
Inside Main Program
4
3
2
1
0
```

```
┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ nano simple_thread.c

┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ gcc -pthread simple_thread.c -o simple_thread.out

┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ ./simple_thread.out
Inside Main Program
4
Inside Thread
0
1
3
2
2
3
1
4
0
```

Answers

1. With `pthread_join()` present, describe the execution order you observed. Which function finished first (thread_function or main's countdown), and why does this ordering occur?

   thread _function finished first because the execution of main's countdown is being blocked by the pthread_join(). The main thread cannot execute its countdown loop until the join is satisfied

2. Without `pthread_join()` (commented out), describe what changed in the output. Did the thread complete its work? Explain what `pthread_join()` does and why the thread's output may be incomplete without it. (Hint: process termination)

   Without the pthread_join(), the main thread does not wait for the thread to finish so they both run concurrently, which makes the output look messy/interleaved. So without pthread_join(), the worker thread may be terminated prematurely

3. Hypothesize what would happen if we had 3-4 threads running similar countdown/countup loops without any synchronization. Would their outputs be interleaved? Explain in terms of OS thread scheduling and context switching.

Yes, Because of OS thread scheduling, the threads share the same CPU. The OS switches between them rapidly. Each thread prints numbers in its own loop. Context switching can occur after any instruction, even in the middle of printing.

Activity 2

Procedure

Read the procedure on the activity file.

```
┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ gcc -pthread thread_args.c -o thread_args.out

┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ ./thread_args.out
Main: Before creating thread
Main: Thread created, continuing with other work...
Thread received: a=10, b=5
Thread computed sum=15
Main: Thread finished. Result: 10 + 5 = 15
```

Answers

1. Explain how data is passed from `main()` to the thread. What is the purpose of the struct `arg_struct`, and why do we pass `&args` (address) rather than args itself?

   So data is passed to the thread by putting all needed values inside struct arg_struct and passing its address to pthread_create(). The arg_struct lets us send multiple values using the single void* parameter that threads accept. We pass &args instead of the struct itself so that both the main thread and the worker thread access the same memory, this allows the thread to modify data that main() can later read

2. Explain how the thread communicates the result back to `main()`. Why can `main()` read `args.sum` after `pthread_join()` returns? What would happen if `main()` tried to read `args.sum` before calling `pthread_join()`?

   So the thread writes the result directly into args.sum using the pointer it was given, which updates the same memory that main() owns. After pthread_join() returns, main() is guaranteed that the thread has finished writing to the struct, so reading args.sum is safe. If main() tried to read

args.sum before calling pthread_join(), the thread might not have computed the sum yet, leading to a race condition and unpredictable output.

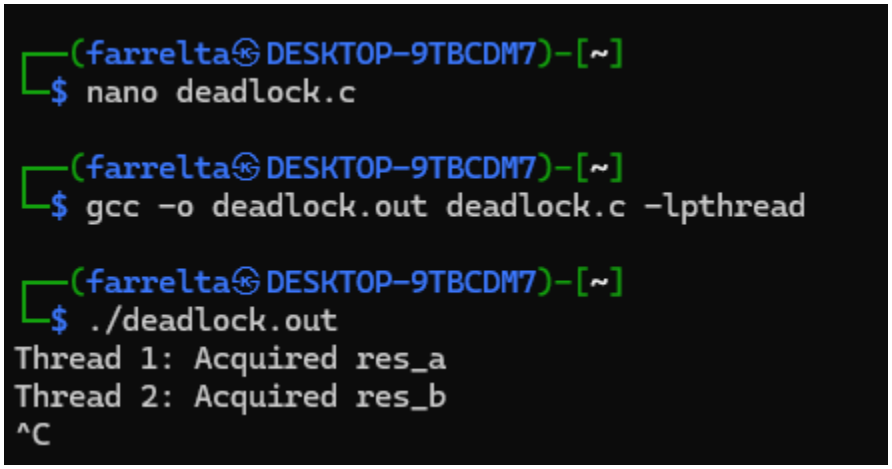
3. The thread includes `sleep(2)` to simulate work. During those 2 seconds, what is `main()` doing? Explain the concurrent nature of this program and how `pthread_join()` synchronizes the two execution flows.

While the thread is sleeping, the main thread continues running until it reaches pthread_join(). At that point, main() blocks and waits for the worker thread to finish. This shows that both threads run concurrently, but pthread_join() serves as a synchronization point that ensures main() does not continue or read shared data until the thread completes its work.

Activity 3

Procedure

Read the procedure on the activity file.



```
┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ cp deadlock.c deadlock_fixed.c

┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ nano deadlock_fixed.c

┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ nano deadlock_fixed.c

┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ gcc -o deadlock_fixed.out deadlock_fixed.c -lpthread

┌──(farrelta㉿DESKTOP-9TBCDM7)-[~]
└─$ ./deadlock_fixed.out
Thread 1: Acquired res_a
Thread 1: Acquired res_b
Thread 1: Released res_b
Thread 1: Released res_a
Thread 2: Acquired res_b
Thread 2: Acquired res_a
Thread 2: Released res_a
Thread 2: Released res_b
Both threads finished
```

Answers

1. Analyze the deadlock. Using the 4 Coffman conditions (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait), demonstrate that all four conditions were present in the original `deadlock.c` program. For each condition, cite specific code evidence.

   The first program satisfies Mutual Exclusion because res_a and res_b are protected by pthread_mutex_lock(), meaning only one thread may hold each mutex at a time (pthread_mutex_lock(&res_a) and pthread_mutex_lock(&res_b)). Hold and Wait is present because each thread locks one resource and then continues running while holding it, attempting to lock another, Thread 1 holds res_a and then waits for res_b, while Thread 2 holds res_b and waits for res_a. No Preemption is enforced automatically by pthread mutexes, since once a thread has locked a mutex, it cannot be taken away forcibly, only the owning thread can call pthread_mutex_unlock(). Finally, Circular Wait occurs because Thread 1 waits for res_b while holding res_a, and Thread 2 waits for res_a while holding res_b, forming a cycle in which each thread waits on the resource held by the other that caused the deadlocks.

2. In the hung state, Thread 1 holds `res_a` and waits for `res_b`, while Thread 2 holds `res_b` and waits for `res_a`. Draw or describe the resource allocation graph showing the circular dependency. Why can neither thread proceed?

In the deadlocked state, Thread 1 holds res_a and waits for res_b, while Thread 2 holds res_b and waits for res_a. The resource allocation graph forms a loop: T1 → res_a → T2 → res_b → T1. Because both threads are waiting on resources that the other will never release, neither one can move forward. Each thread is blocked forever, creating a complete standstill.

3. Explain how ordering the resource requests identically in both threads (Part B) broke the circular wait condition. Which of the 4 Coffman conditions did we eliminate, and why does eliminating just one condition prevent deadlock?

In the fixed version, both threads acquire the locks in the same order, res_a first, then res_b. This breaks the circular-wait condition because a cycle can't form if every thread follows the same ordering rule. The other three Coffman conditions still exist, but removing just one is enough to prevent deadlock, since deadlock requires all four.

4. Beyond resource ordering, name and briefly describe two other deadlock prevention strategies from the module (e.g., eliminating hold-and-wait, using timeouts, banker's algorithm). For each, explain which Coffman condition it targets.

One common strategy is to eliminate "hold" and "wait" by requiring threads to request all their needed resources at once; this removes the chance that a thread holds something while waiting for something else.
Another is to use timeouts or try-lock operations, where a thread backs off and releases what it holds if it can't get the next lock quickly. This breaks the no-preemption condition by letting the system "reset" before a deadlock forms.